2026-05-08-p4-test-challenge-hardening

Phase 4 — Test/Challenge Hardening — Implementation Plan

> **Spec:** `docs/superpowers/specs/2026-05-08-helixcode-zero-bluff-completion-design.md` §6
> **Previous phase:** Phase 3 complete (`1f1d8f4`)

Goal: Verify every test and challenge carries positive runtime evidence. Eliminate all remaining bluff patterns. Ensure CONST-035/Article XI §11.9 compliance across the entire test suite.

File Structure Map

```
# New files to create
helix_code/internal/anti_bluff/types.go              — create (bluff taxon
helix_code/internal/anti_bluff/scanner.go            — create (grep + stat
helix_code/internal/anti_bluff/verifier_test.go      — create (self-test)
helix_code/tests/e2e/challenges/challenges/anti-bluff-verifier-001.json —

# Files to audit/modify
helix_code/internal/*/ *_test.go                     — audit all test file
helix_code/tests/e2e/challenges/                     — audit all challenge
helix_code/cmd/                                      — audit test files
```

Task P4-T01: Full Test Suite Audit

Goal: Classify every test file, identify skip/silent-pass/no-evidence patterns.

Steps: - [ ] Scan all `*_test.go` files under `helix_code/internal/`, `helix_code/cmd/` - [ ] Count by type (unit, integration, e2e, benchmark) - [ ] Identify `t.Skip()` without `SKIP-OK:` marker - [ ] Identify tests that assert on metadata-only (e.g., `assert.NotNil`) without runtime evidence - [ ] Identify tests with `testMode: true` or `mock` in non-unit contexts - [ ] Classify all tests for bluff risk (high/medium/low)

```
cd HelixCode
# Count test files and functions
find . -name '*_test.go' | wc -l
grep -rn "^func Test" ./internal/ ./cmd/ | wc -l
# Mark all tests with their type
```

Task P4-T02: Challenge Harness Audit

Goal: Verify every challenge produces positive runtime evidence (not just exit code 0).

Steps: - [ ] Audit `tests/e2e/challenges/challenges/*.json` for assertion quality - [ ] Verify each challenge has `expected.json` with sha-256 or equivalent verifiable evidence - [ ] Verify exit-code logic is not masking failures - [x] Run a representative sample of challenges against real providers

```
cd helix_code/tests/e2e/challenges
# List all challenges
ls challenges/
# Run challenge runner with --dry-run to validate structure
```

Update 2026-05-08 22:07: Permissions challenge PASS after binary rebuild. Old binary (May 8 00:01) lacked dispatcher code. Binary rebuilt with `go build -tags nogui -o bin/helixcode ./cmd/cli`. All 3 scenarios verified with runtime evidence.

Task P4-T03: Bluff Taxonomy Sweep

Goal: Find every instance of the 5 bluff patterns across test and challenge files.

Steps: - [ ] Wrapper bluff: check `assert.Equal(t, 0, exitCode)` where exitCode is always 0 - [ ] Contract bluff: check for `t.Skip("feature not available")` without `SKIP-OK:` - [ ] Structural bluff: check file-exists tests that don't verify content - [ ] Comment bluff: grep `grep "for now\|TODO implement\|placeholder\|simulated"` in test files - [ ] Skip bluff: all bare `t.Skip()` without `SKIP-OK:` marker

```
cd HelixCode
# Anti-bluff sweep on test files specifically
grep -rn "simulated\|for now\|TODO implement\|placeholder\|stub"
        internal/ cmd/ --include='*_test.go'
# Skip audit
grep -rn "t\.Skip(" internal/ cmd/ --include='*_test.go' | grep
        -v "SKIP-OK"
```

Task P4-T04: Fix All Identified Bluffs

Goal: Every test and challenge that passes must produce positive runtime evidence.

Fix patterns: - Replace `assert.NotNil(t, result)` with `assert.NotEmpty(t, result.Content, "response must have content")` - Replace `assert.NoError(t, err)` with verification of actual returned data against expected values - Add `assert.Greater(t, len(models), 0)` to model listing tests - Add real HTTP response content assertions to provider tests - Ensure every test's pass means "this feature works for a user"

Task P4-T05: Anti-Bluff Verifier Challenge

Goal: Automated challenge that scans for forbidden patterns and fails if any found.

Challenge specification:

```
{
  "name": "anti-bluff-verifier-001",
  "description": "Verify zero bluffs in production and test
      code",
  "checks": [
    "grep simulated fails in non-test code",
    "grep placeholder fails",
    "grep 'TODO implement' fails",
    "all t.Skip() have SKIP-OK marker",
    "no test asserts only on nil/not-nil without content
      verification"
  ]
}
```

Task P4-T06: Challenge Coverage Gaps

Goal: Ensure every package has at least one challenge that verifies end-user functionality.

Steps: - [ ] Map existing challenges to packages - [ ] Identify packages with zero challenge coverage - [ ] Create minimal challenges for uncovered packages

---

Task P4-T07: Full Infrastructure Test Run

Goal: Run the complete test suite against real infrastructure (docker-compose.full-test.yml).

Prerequisites: Docker/Podman running.

```
cd HelixCode
make test-infra-up
make test-complete
make test-infra-down
```

Task P4-T08: Cross-Compile Verification

Goal: Verify builds on all target platforms.

```
cd HelixCode
GOOS=linux GOARCH=amd64 go build ./...
GOOS=darwin GOARCH=arm64 go build ./...
GOOS=windows GOARCH=amd64 go build ./...
```

Governance Verification (parallel)

Before code changes, verify anti-bluff anchor is present in:

1. Root governance files: CONSTITUTION.md, CLAUDE.md, AGENTS.md — DONE (Phase 1)
2. Inner HelixCode AGENTS.md — DONE (Phase 1)
3. All owned-by-us submodules: HelixAgent, HelixQA, Challenges, Containers, Security, dependencies/HelixDevelopment/* — VERIFY
4. All third-party submodules — MUST add `.helix-governance` marker if missing

```
./scripts/verify-governance-cascade.sh
```

Anti-Bluff Verification Layer (mandatory before Phase 4 close-out)

1. `grep -rn "simulated\|placeholder\|stub\|TODO\|for now"` — zero hits in non-test production code
2. `grep -rn "t\.Skip("` — all matches have `SKIP-OK:` marker
3. `go build ./...` exits 0
4. `go vet ./...` exits 0
5. `go test -short ./...` exits 0
6. All applicable challenges pass with runtime evidence
7. `verify-governance-cascade.sh` exits 0